Three Bit MLP Vector Quantization in Apple Neural Engine

Anemll Open Source Project

www.anemll.com | realanemll@gmail.com

13 October 2026

Code: https://github.com/anemll/anemll-forge [25]

Abstract

Deploying large language models on the Apple Neural Engine requires quantization that matches its lookup-table layout and compiler behavior. We present ANEMLL Forge release 0.2, an adaptation of Qwen3.8-27B that represents every MLP matrix with a shared 64-entry codebook of two-component vectors and per-output-channel scales, giving three index bits per weight. The method combines online Hadamard rotations, GPTQ, rank-64 corrections on token-mixer projections, and 600 steps of quantization-aware training with frozen indices. Inference uses INT8 attention scores and FP8 softmax intermediates on M6, with FP16 intermediates on M5. Using identical preceding tokens for both models, we evaluated the compiled M6 model against the BF16 reference on 64 recorded conversation sequences (40,023 next-token positions), obtaining mean KL-512 of 0.0518 and 91.9% top-1 agreement. M6 and M5 Max KL values are within 0.0003 of the same-weight PyTorch result. Packed weights require 9.74 GiB including correction factors, excluding embeddings, equivalent to 3.27 bits per base weight. Compiled target memory is 12.97 GB. With speculative decoding, the M6 averages 37.5 tokens/s over 24 chat and coding prompts. These results measure distributional fidelity and workload-specific performance; task accuracy and energy efficiency remain unevaluated.

1 Introduction

Low-bit weight storage is useful only if the deployed representation preserves both its storage savings and acceptable model fidelity. On the Apple Neural Engine (ANE), vector lookup tables impose choices about codebook size, vector direction, scale placement and compilation. This work studies those choices for the Qwen Team's Qwen3.8-27B model [24]. ANEMLL contributes the quantization, conversion and runtime implementation [25, 26].

Release 0.2 uses three-bit vector lookup tables for all MLP matrices and extends online Hadamard rotations to the token mixers. GPTQ initializes the representation, followed by quantization-aware training (QAT). The model runs through Apple Core AI and a Swift bridge on macOS 27, with separate attention acceleration described in Section 4. The contribution is the measured combination of these established methods under the tested ANE constraints:

- A shared 64-entry, two-component codebook for each MLP matrix, with vectors along output channels. Each vector stays within one input column, so GPTQ's column-wise error feedback applies without crossing vector groups.
- A format choice based on compiled memory and measured execution time, accounting for index storage, compiler-dependent weight sharing and chip-specific attention precision.
- An evaluation of a hybrid Gated DeltaNet and full-attention model on M6 and M5 Max, including same-weight PyTorch comparisons, quantization ablations and explicit limits on the quality and throughput conclusions.

The release 0.1 baseline used two-bit vector tables in 38 MLP layers and four-bit scalar tables in 26. Later experimental baselines also use mixed formats, but differ from release 0.1 in fitting and allocation; they are identified separately below [26].

2 Related work

GPTQ [1] supplies error-compensating rounding. GPTVQ [2] extends GPTQ to vector codebooks, while AQLM [3] and VPTQ [4] study additive and vector quantization for language models. QuIP [5] and QuIP# [6] use randomized transforms to improve quantization; QuaRot [7] and SpinQuant [8] apply rotations throughout the network. SmoothQuant [9] motivates the separate input-channel rescaling ablation tested here. Weighted codebook fitting appears in SqueezeLLM [10], and differentiable k-means palettization in DKM [11].

Low-rank compensation follows ZeroQuant-V2 [12] and LoftQ [13]. Optimizing continuous quantization parameters against a teacher with fixed codes is related to AQLM, PV-Tuning [14] and EfficientQAT [15]. LLM-QAT [16] likewise uses teacher-generated text, building on knowledge distillation [17]. Dutta et al. [18] motivate evaluating distributional fidelity and top-1 agreement in addition to task accuracy. Apple's foundation language models [19] use mixed two- and four-bit palettization with accuracy-recovery adapters; the convolution-based layout here follows Apple's guide to transformers on the ANE [23].

Low-precision attention is also studied by SageAttention2 [52], which quantizes query/key operands to INT4 and probability/value operands to FP8 on GPUs. Our attention contribution concerns compiler-fused INT8 and FP8 dataflow across the ANE's tiled softmax passes and its M5/M6 precision variants, as detailed in Section 4 [49, 50].

Hardware format constraints

Fine-grained block scaling helps low-bit formats accommodate local changes in weight magnitude. MXFP formats share a scale across 32 values; MXFP4 uses four-bit floating-point elements and an eight-bit power-of-two scale [38]. NVIDIA's NVFP4 uses 16-value blocks with FP8 scales and an additional FP32 tensor scale [39]. Each block can therefore fit its own dynamic range instead of sharing a scale across an entire row or matrix.

On the tested ANE path, the lack of MXFP4/NVFP4-style block-scaled execution is a major constraint on low-bit quantization. Core ML's per-block integer quantization is optimized for GPU execution; Apple recommends per-channel scales for the ANE [40]. Framework-level support must therefore be distinguished from ANE support. Grouped scalar palettization can run on ANE, but uses multiple lookup tables rather than block-scaled floating-point values [40, 41]. In our probes, grouped vector tables fell back to CPU [41]. The working vector path uses one shared codebook per matrix and per-output-row scales; rotations and QAT recover fidelity within these constraints.

Vector quantization provides a working ANE-accelerated route on both M5 Max and M6. The repository's matched convolution sweep reports ANE placement on both systems and substantially lower lookup-table latency on M6 (Table 3) [42]. The selected release table contains 64 two-component centroids, or 128 values, within the 256-value boundary observed for the tested vector-LUT path [26, 41]. Hardware acceleration here means execution assigned to the ANE; the sweep alone does not establish the internal decoder design. Storage and execution behavior are assessed separately.

3 Method

Model and vector representation

The model has 64 layers: 48 Gated DeltaNet layers [20] and 16 full-attention layers. Hidden and MLP intermediate widths are 5,120 and 17,408. Each MLP has gate, up and down matrices, for 192 matrices in total. Figure 1 shows their placement in the residual block. The released allocation is recorded in a versioned configuration [29].

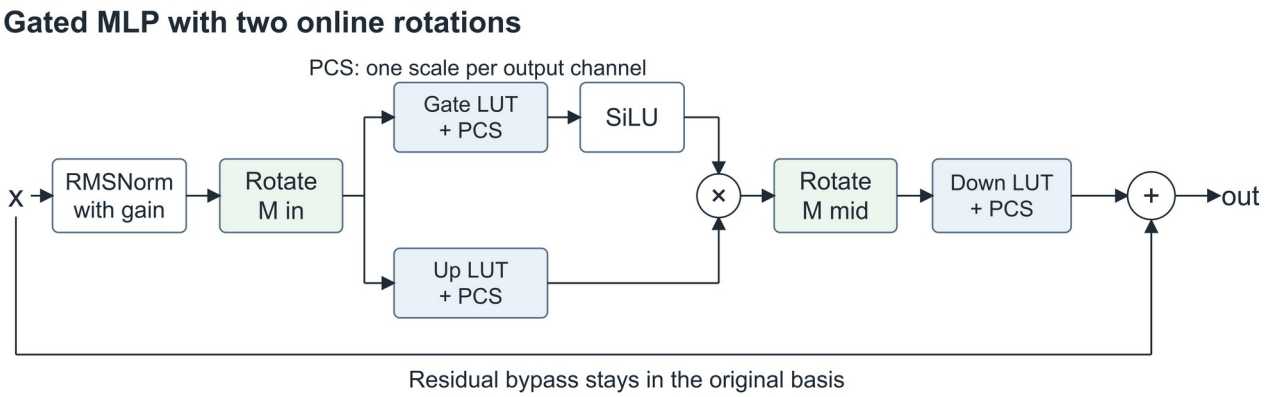


Figure 1. The MLP path. Gate and up share an input rotation; down receives a separate rotation after elementwise gating. Each blue projection includes a vector lookup table and per-channel scaling (PCS), with a separate scale for every output row. Gate scaling precedes SiLU, up scaling precedes elementwise gating, and down scaling precedes the residual addition. Green boxes are signed, normalized Hadamard rotations in blocks of 1,024. RMSNorm keeps its learned gain, and the residual bypass keeps its original basis. No rotation is moved through SiLU. M in and M mid denote the input and intermediate transforms; seeds are 1000 + layer and 2000 + layer. Implementation: the LayerW, Hadamard and QConv components in [28].

All 192 gate, up and down matrices use vector 2x64 + pcs (pcs: per-channel scaling). Each matrix has 64 learned two-component centroids. One six-bit index represents two consecutive output channels at the same input column. Each output channel has its own scale. The index payload therefore costs three bits per weight, with additional codebook and scale storage. Figure 2 illustrates the vector lookup and row-scale reconstruction [27, 28, 29].

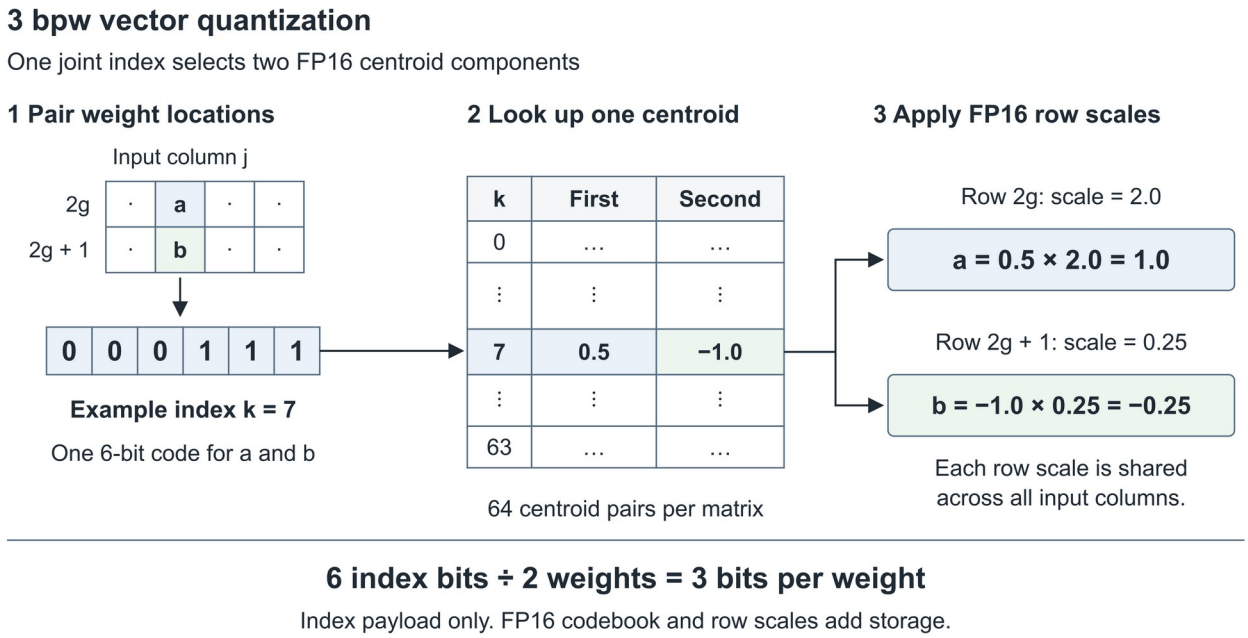


Figure 2. Three-bit vector quantization with per-output-channel scales. A single six-bit index selects both FP16 components of one centroid; they represent adjacent output rows at the same input column. The illustrative index 7 selects (0.5, −1.0). Row scales (2.0, 0.25) reconstruct weights (1.0, −0.25). Each scale is shared across its whole output row. Six index bits cover two weights, giving three bits per weight for the packed index payload. Each matrix also has a shared 256-byte FP16 codebook and two bytes of scale per output row; compiled table copies and other runtime storage are additional. This representation does not imply INT3 arithmetic [27, 28].

The safetensors export stores one uint8 index per weight pair, so its index payload occupies four bits per weight. Conversion packs four six-bit indices into three bytes, with codes crossing byte boundaries. The codebook and row scales are separate.

The Core AI builder imports the exported tables and indices instead of fitting replacement codebooks. For LUT weights, per-output-channel scales multiply the convolution output before the next nonlinearity or residual addition; palettization runs before optimization so scale folding does not destroy the shared table structure. Three-bit describes MLP weight indices. Activations, recurrent state and the KV cache have separate runtime precision choices [28].

Per-channel scaling across model blocks

Per-channel means one scale per output row, shared across that row's input weights. All 192 MLP matrices use it. The selected token-mixer LUT projections also use it: DeltaNet in_proj_qkv, in_proj_z and out_proj, and full-attention q_proj and o_proj. Their codebooks are vector 2 × 16 in layers 0–23 and scalar LUT4 in layers 24–63. The LM head uses scalar LUT4 with one scale per output channel, corresponding to a vocabulary logit [26–29].

Full-attention k_proj and v_proj instead export symmetric INT8 codes and one FP16 scale per output channel. The default Core AI builder preserves these codes as compile-time INT8 weights with FP16 scales; an optional compatibility setting expands them to FP16. Compiled-program inspection confirms INT8 projection kernels on M6, as discussed in Section 4. This format concerns projection weights, not the KV cache. Rank-64 correction factors, embeddings, normalization parameters, DeltaNet's small in_proj_a/in_proj_b projections and depthwise-convolution weights remain floating point; they do not use these quantization scales. For LUT mixers, the scaled main projection and the FP16 low-rank correction are added as separate branches [26–28, 49, 50].

Online rotations

The MLP already used online, normalized Hadamard rotations in 1,024-wide blocks with seeded signs. Gate and up share an input rotation; down receives a separate rotation after the nonlinear gated activation. V0.2 extends this approach to the large DeltaNet [20] and full-attention input and output projections. The residual stream, embeddings and LM head keep their original basis [26, 28].

For row-vector input x and an orthogonal transform M, rotate the input to xM and the weight to WM. Their product remains $xMM^TW^T = xW^T$ before quantization. Quantizing WM changes the approximation, while the activation rotation remains online. The export records block width and seeds, and the builder represents the transform as a grouped convolution with exact values +1/32 and −1/32 [28].

Mixer input projections use seed 3000 + layer, and output projections use 4000 + layer. The large DeltaNet and attention projections are rotated; the small DeltaNet in_proj_a and in_proj_b paths are unchanged. Rank-64 correction factors are stored in their original basis, and the converter moves their input factor into the rotated basis. These conventions must agree between the export, reference evaluation and compiled graph [26, 28].

GPTQ workflow

GPTQ uses calibration activations to compensate for rounding error. Here the pass is sequential, with rank-64 mixer residual factors fitted inside the pass, as in LoftQ [13]: GPTQ quantizes W − AB, and the calibration stream continues through Q + AB, so later layers see the quantized outputs of earlier ones. The v0.2 run used 128 calibration rows of 1,024 tokens drawn from WikiText (44 rows), BF16 chat traces (36) and rendered coding-agent sessions (48). The sequential quantizer and LUT helpers are public [27], with a complete training workflow in [45].

Figure 3 separates three operations: measuring input-channel sensitivity, fitting the centroid table, and choosing indices with GPTQ error feedback. PCS first divides each output row by its FP16-rounded RMS scale, replacing zero scales with one. Adjacent rows at the same input column then form two-component samples for one shared codebook. A centroid supplies a reusable pair of normalized values; two independent row scales restore their magnitudes. For example, centroid (0.5, −1.0) with row scales (2.0, 0.25) reconstructs weights (1.0, −0.25). This example is illustrative [27].

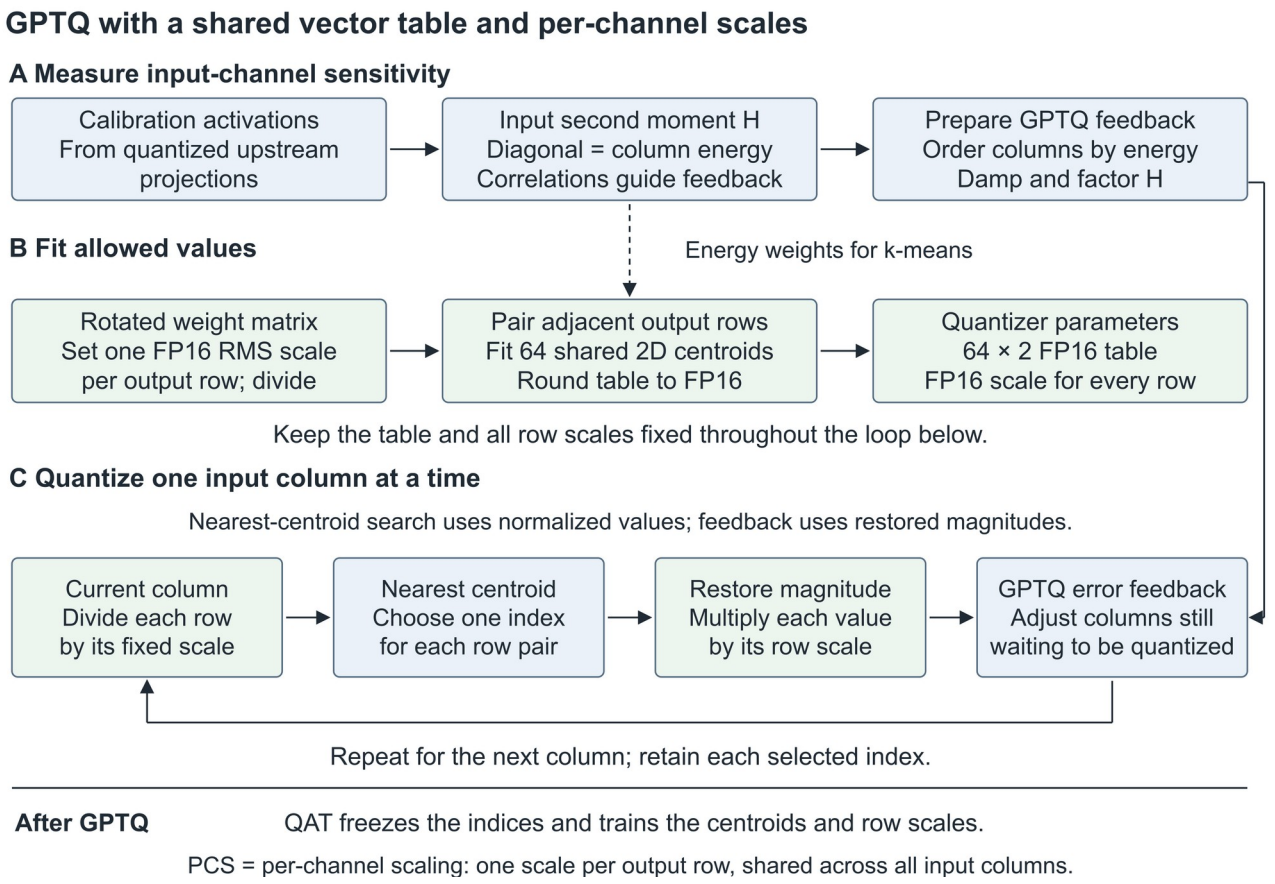


Figure 3. GPTQ for an MLP matrix using vector 2 × 64 with per-channel scaling. Calibration forms H by averaging input outer products; its diagonal weights centroid fitting and orders columns, while its damped inverse factor drives error feedback. The vector rounder divides the current column by row scales, selects the nearest centroid in normalized space, and restores row magnitudes before computing the error. Each vector lies within one input column. The table and scales stay fixed throughout this loop; QAT later trains both with indices frozen. Mixer low-rank fitting adds an outer loop that can refit the table and scales between GPTQ calls [27, 45, 46].

Calibration enters twice. In the published recipe, activation-weighted k-means gives each input column a weight from diag(H), so columns with larger calibration energy contribute more to centroid fitting. Vector fitting uses at most 300,000 sampled pairs, three initializations and up to 200 iterations. Scalar tables use 40 weighted Lloyd updates on at most 65,536 values per table. Centroids are rounded to FP16 before index selection. GPTQ then visits input columns in descending energy order, adding 1% of the mean diagonal of H to every diagonal entry and grouping updates into blocks of 128 columns [27, 45].

PCS and GPTQ act on different axes: PCS normalizes output rows, while GPTQ compensates error across input columns. The implementation chooses centroids by Euclidean distance after dividing by the row scales. It restores the scales before computing the error passed to later columns; for unequal row scales, the nearest centroid in normalized space need not minimize unscaled weight error. The 128-column processing blocks organize computation and introduce no additional block scales or codebooks. Within the MLP, down-projection calibration uses the already-quantized gate and up outputs [27].

The mixer factors add a low-rank correction through two thin FP16 projections alongside each selected quantized projection. They carry storage and compute costs (0.30 GiB of FP16 for the whole model). The MLP receives no such factors: the local three-bit tests found only a 2.8% error reduction from rank-64 MLP corrections [26, 29].

Quantization aware training

QAT distills the BF16 model's next-token distribution. Indices and INT8 codes stay frozen while lookup-table values, per-channel scales, mixer factors and head parameters train, about 162 million values in total. Each scale is its GPTQ value multiplied by $\exp(\delta)$, with δ initialized to zero; INT8 K/V projection scales train as well. The LM head's table and scales form a separate optimizer group. Trained tables, scales and factors are rounded to FP16 on export. The loss uses the teacher's top 256 tokens plus an aggregate tail bucket. Historical teacher log-probabilities are cached in FP16 and the KL loss is computed in FP32, so cache rounding introduces a small numerical bias. Figure 4 separates fitting, conversion and inference [26, 46].

The QAT corpus contains BF16 responses, including thinking, to 527 prompt entries: 141 code, 130 tool-use, 126 multilingual, 50 math and 80 rare-token prompts [47]. These entries contain 515 distinct prompt strings. Historical filtering removed 33 entries from a 560-candidate pool, with word 8-gram overlap and near-duplicate checks reported in [26]. The resulting corpus has 673 rows of 1,024 tokens, including 637 training rows. Sequence-level splitting separates generated sequences, but repeated prompts mean that prompt-level uniqueness across splits is not guaranteed.

The documented teacher recipe combines the 637 corpus training rows with 44 coding-agent, 32 chat and 44 WikiText rows, giving 757 training rows. It reserves four rows from each source, or 16 rows, for monitoring. A separate 128-prompt selection trace chooses the checkpoint. Training uses Adam, batches of two rows, a 20-step warm-up, cosine learning-rate decay and gradient-norm clipping at 1.0. Learning rates are 2×10^{-4} for tables and scales, 2×10^{-5} for mixer factors and 1×10^{-4} for the head. Selection runs every 50 steps; the selected checkpoint was step 600 [26, 45, 46].

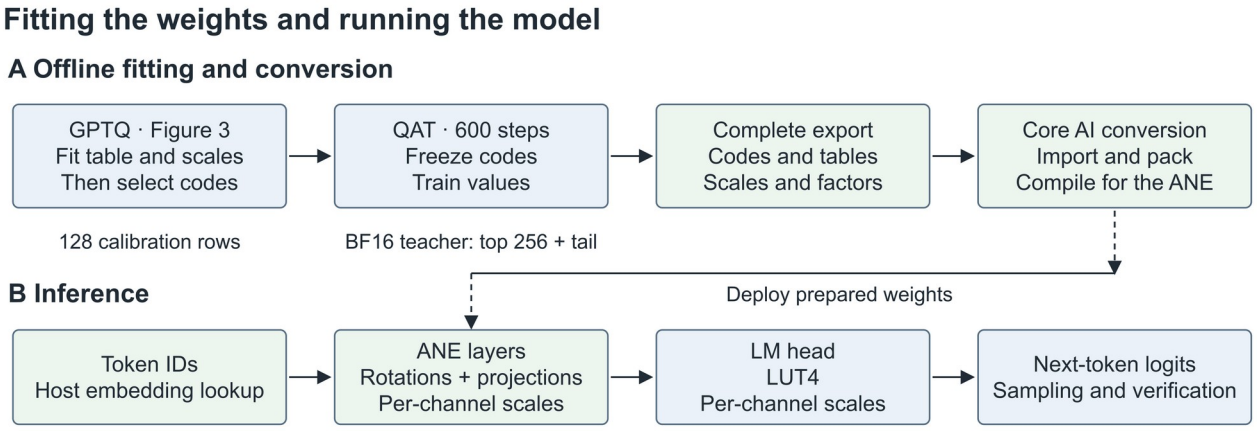


Figure 4. Offline fitting and deployed inference. GPTQ produces indices and continuous quantization parameters. QAT updates the continuous parameters against a cached BF16 distribution while keeping indices fixed. Materialization produces a complete export with rotation metadata; Core AI imports exact tables and indices and palettizes before optimization. At inference, the host performs FP16 embedding lookup, followed by compiled target layers and the output head. Both LUT and INT8 projection weights use per-output-channel scales. QAT also updates the mixer correction factors and head parameters. The prepared bundle also contains the matching DFlash2 drafter. Cache and recurrent-state management, final normalization and the speculative sampling loop are omitted for clarity [26, 28, 34, 46, 48].

4 Accelerated inference on the ANE

Weight fitting and runtime attention precision are separate parts of the implementation. GPTQ fits the K/V projection weights to signed INT8 codes with FP16 per-output-channel scales. During inference, the graph

additionally quantizes attention scores and softmax intermediates, and uses a value cache stored in INT8. These runtime choices operate on changing activations and cache entries; they are not fitted by the weight-only GPTQ pass [27, 28, 49, 50].

The default V8 cache quantizes only V: historical keys remain FP16 and values are INT8, with FP16 scales per token and KV head. The converter also supports an optional KV8 export with both keys and values in INT8. Its option named “both” instead packages selectable FP16 and V8 cache layouts. Runtime selection requires matching exported functions. Both K and V projection weights use INT8 independently of these cache choices [27, 28, 57].

INT8 projection kernels and W8A8 compute

The default K/V projections use FP16 activations and compile-time INT8 weights with FP16 channel scales. The compiler retains INT8 weight kernels without an FP16 copy; the compatibility option QCONV_INT8=0 expands them to FP16. The recorded M6 inspection found 50 to 40 MB of compiled weights per attention layer and 272 to 152 static cycles for one 64-row projection task. These estimates do not establish whole-layer latency [28, 50].

Quantizing activations as well as weights (W8A8) enables faster INT8 × INT8 arithmetic. Apple documents this ANE path for M4-class hardware [40]. Our M6 probe measured 38.4 TOPS with per-channel weight scales versus 18.6 for FP16 (2.06×); shared scales reached 52.8 TOPS. These are isolated 4,096 × 4,096 convolution results with 1,024 rows per call, not a matched M4/M5/M6 comparison. Supported quantize/dequantize pairs fuse into producer and consumer tasks, preserving INT8 operands without separate FP16 conversions. The release projections do not insert this activation quantization; W8A8 was tested separately. The selected M6 attention core uses FP16 × FP16 for QK and INT8 × FP8 for cached-value weighting, as the recorded HWX confirms [28, 49, 50].

FP8 softmax intermediates on M6

The M6 path keeps the query–key dot product in floating point but writes its scores in INT8, using a quantization step of 0.25. Quantize/dequantize boundaries also keep masked scores in INT8. Figure 5 follows the resulting softmax calculation. For each history tile, exp(score − tile maximum) is stored in FP8 E4M3 before forming the normalization sum (sm8). The same exponential weights then incorporate the per-token value-cache scales and are stored in FP8 again before multiplying the INT8 cached values (pvf8). These are two stages of one softmax path. The weights remain unnormalized until tile results and the current floating-point block are combined and divided by the denominator [28, 49].

The acceleration comes from keeping large score and exponential tensors at one byte per element across compiler-fused passes. The inspected program has no dedicated hardware softmax instruction, and its maximum, sum and final attention outputs are FP16. Thus “FP8 softmax” describes the intermediate dataflow, not an entirely FP8 calculation. The measured INT8-by-FP8 product has the same multiply-add cycle count as its FP16-operand control; the benefit there is reduced data movement. FP8 also retains much more small-probability mass than the tested uniform UINT8 alternative after the value-scale fold [49, 50].

Table 1. Precision of the accelerated cached-history attention path. Both variants use the same fitted projection weights. K/V projection-weight scales are per output channel; V-cache scales are per cached token and KV head within each attention layer. The current query block retains its floating-point path. “Reduction output” states the observed tensor format, not undocumented internal accumulator precision [28, 49–51, 57].

Stage or tensor	M6	M5
Query–key multiply operands	FP16 × FP16	FP16 × FP16
Scores after quantization and mask	INT8	INT8
Exponentials used by the softmax sum	FP8 E4M3	FP16
Maximum and sum reduction outputs	FP16	FP16
Scaled softmax weights for cached V	FP8 E4M3	FP16
Stored history keys / values	FP16 / INT8	FP16 / INT8
Attention output	FP16	FP16

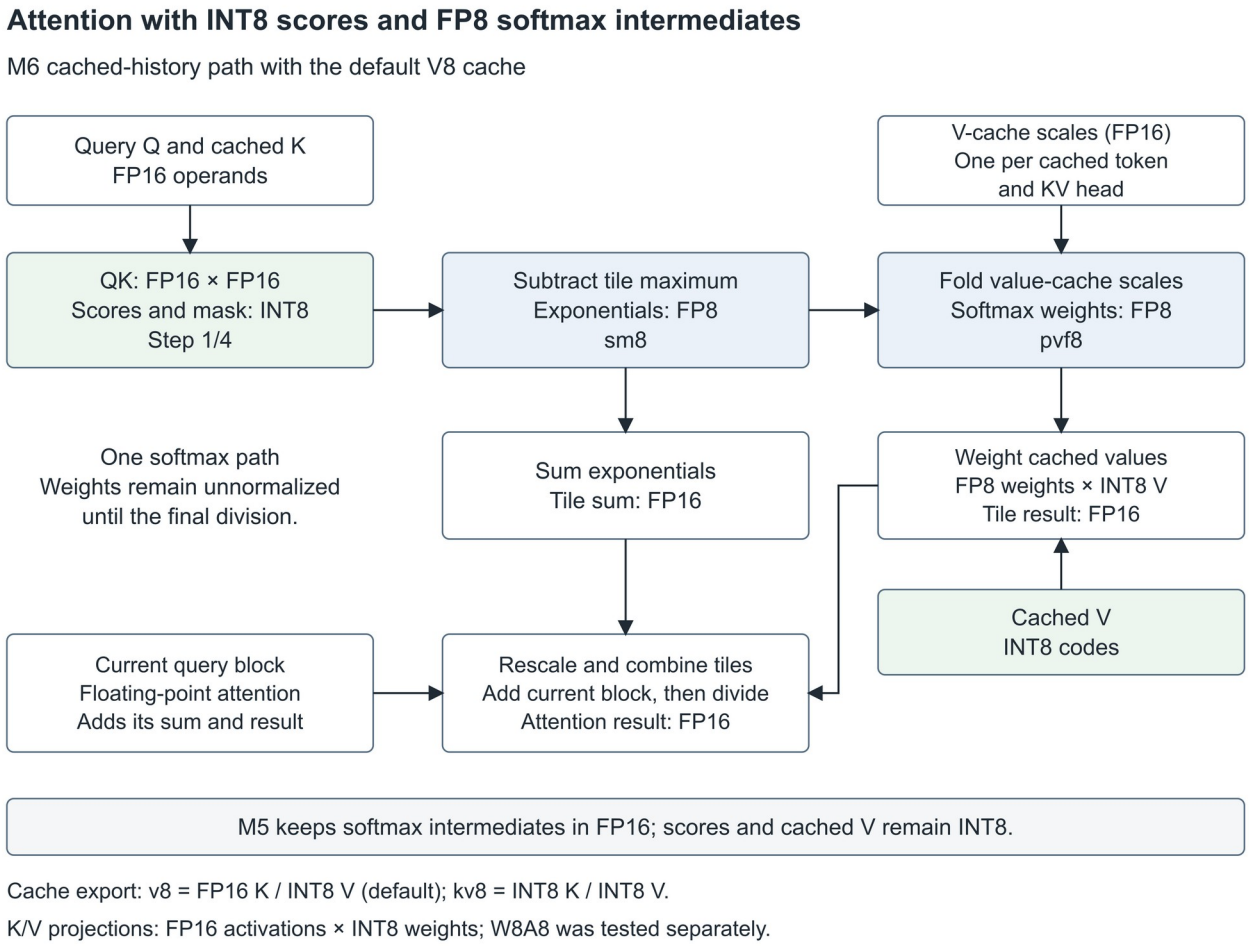


Figure 5. M6 tiled attention with FP16 QK operands, INT8 scores, FP8 softmax intermediates and INT8 cached values. Within each attention layer, one FP16 V-cache scale is stored per cached token and KV head, shared across that value vector. The normalization sum branches before these scales enter the value-weighting branch. Partial results restore scale factors, combine with the current floating-point block, and normalize. Maxima, sums and attention results have FP16 output tensors; internal accumulator precision is not established. M5 keeps softmax intermediates in FP16. V8 quantizes V only; optional KV8 also quantizes K. K/V projections use FP16 activations and INT8 weights. The attention gate and output projection are omitted [28, 49, 57].

Range control and the M5 path

FP8 range needs an additional implementation constraint. A scale of 1/256 produced incorrect M6 results for one captured attention head when a scaled softmax weight reached about 200. The selected scale is 1/64: intermediates bounded to [0, 1] reach FP8 values of at most 64 after scaling. This bound does not mean the weights sum to one. Device output then matched the host simulation in the investigated case. This is an observed implementation limit, not the representable range of E4M3. The tested M5 ANE compiler rejects the FP8 variant. Its functions retain INT8 scores and transposed FP16 keys while keeping softmax intermediates in FP16. Release 0.2 validation confirms ANE placement for the M5 function set [28, 50, 51].

Release 0.1 did not use FP8 attention. Later experiments kept the earlier weights fixed while comparing a non-FP8 V8 graph with the combined INT8-score and range-corrected FP8 graph (C2), before transposing keys. Target-only prefill rose from 292.5 to 310.8 tokens/s at 8K and from 199.6 to 223.9 tokens/s at 64K; the 64K direct KL to the V8 control was 0.00048. These gains measure an experimental graph change, not a release-to-release comparison or FP8 alone. The later transposed-key layout removes repeated key-tile transposes [50].

Blocked Gated DeltaNet verification

DFlash2 proposes seven tokens after an anchor, and the target verifies all eight rows together. In the Gated DeltaNet layers, each row depends on earlier rows. The established chunkwise delta-rule formulation resolves these dependencies through a small triangular system [20, 56]. For an eight-row block, its strictly lower-triangular matrix N satisfies $N^8 = 0$, so the inverse is a finite product:

$$(I + N)^{-1} = (I - N)(I + N^2)(I + N^4).$$

The builder replaces seven serial row updates with this product and one multiplication by the right-hand side. Expressing the small products as broadcast multiplication followed by reduction keeps the operation on the ANE; direct chains of computed matrix multiplications failed ANE compilation in the recorded experiment. The enabled GDN_FAST path also uses native depthwise convolution and combines two state products into one per prefill sub-block [28, 50].

In the historical M6 microbenchmark of three Gated DeltaNet cores, excluding their input/output projections, this combination reduced eight-row verification from 1.18 to 0.64–0.69 ms and 64-row prefill from 6.82 to 3.42–3.49 ms. Recorded device output error was unchanged at the reported precision. The algebraic rewrite is exact; floating-point execution can still differ. These are core timings, not full-model speedups. Single-token recurrence has no block solve and does not receive this particular benefit [50].

Accepted state and drafter execution

Recurrent updates from the current verifier block remain pending until the accepted length is known. The next target call commits only that prefix, and the runtime appends only accepted K/V rows. Rejected proposals do not advance committed state. Persistent buffers and reusable Swift binding plans limit allocation and dispatch overhead. Full-attention history tiles default to 2,048 rows for verification and 4,096 for prefill, following separate tile-size experiments [28, 50, 54].

Measured full-target calls at 64K took 131.5 ms for eight rows, 127.2 ms for four and 123.1 ms for three. Combining these timings with recorded draft acceptance predicted 21–36% lower throughput for shorter blocks on the coding-agent workload. This is an acceptance-based model with fixed non-verifier overhead, not an end-to-end measurement of shorter verifiers; package entry sets also differed. It supports the eight-row choice for that workload without assuming eight emitted tokens per call [53].

The quantized Core AI drafter fuses up to eight new target-feature rows with the next draft block in one call. Its draft and context-update entries share weights, while the host manages a 2,048-slot K/V ring. During prompt prefill, the server ingests only the final 2,056 feature rows before each cached-prefix snapshot boundary and the prompt end; earlier drafter rows would be overwritten before use. The target still processes

every prompt token. The full target vocabulary head is retained, followed by the upstream rank-256 selector over 16 candidates per draft position. Rebuilding the drafter with the target's QAT-updated head preserves the intended pairing [26, 54, 55].

5 Experimental setup

Hardware and artifacts

Compiled-model fidelity and runtime measurements use the M6 and M5 Max ANE paths. PyTorch quantization comparisons use M3 Ultra, while fitting uses one Google Colab G4 instance with an NVIDIA RTX PRO 6000 Blackwell GPU and 96 GB of memory. The inference path requires macOS 27 and a compatible Xcode 27/Core AI SDK. The environment guide [31] distinguishes inference, conversion and historical research environments; these should not be treated as one interchangeable software stack.

GPTQ is implemented in PyTorch with Hugging Face Transformers for calibration forwards. Initial vector centroids use scikit-learn k-means on CPU, and low-rank SVD fitting also runs on CPU. On Colab, calibration, Hessian factorization and GPTQ error compensation use CUDA; QAT also uses CUDA. The GPTQ script supports Apple GPU execution through MPS, with FP64 Hessian factorization on CPU. The recovered QAT workflow has not been validated end to end on MPS [27, 45, 46].

Producing the released weights took about four GPU-hours on one Google Colab G4 instance with an NVIDIA RTX PRO 6000 Blackwell and 96 GB: 54 minutes for GPTQ, three minutes for the teacher cache, and 3.1 hours for 600 QAT steps, including periodic evaluation. Generating the BF16 calibration chats and the QAT corpus is not included. The experiments that selected the recipe cost another roughly 28 G4-hours and four A100-hours. The approximately four-hour figure covers the final fitting run, not the complete research effort [26].

The upstream BF16 checkpoint [24], quantized export [33], prepared inference bundle [34] and GitHub source [25] are identified by revisions. Those revision identifiers establish artifact identity; they do not replace complete per-run logs of compiler builds, prompts, seeds and sampling settings. Section 8 records the remaining reproduction limits.

Evaluation protocols

We evaluate compiled fidelity on 64 recorded conversation sequences, covering 40,023 next-token positions across prompts and answers. Teacher forcing feeds both models the same recorded preceding tokens at each position, so their predictions are compared using identical contexts. KL-512 is KL(BF16 || quantized) after partitioning the vocabulary into the BF16 teacher's top 512 tokens and one aggregate tail bucket. It is averaged over scored positions in nats; lower values mean closer distributions. Top-1 agreement is the fraction of positions at which both models select the same highest-probability token. The released evaluator computes log-softmax in FP64 and floors aggregate tail probabilities at 10^{-12} [30].

The cross-format comparison in Table 2 uses a different protocol: the shared 45-token system prompt is excluded, and development and held-out chat traces are scored separately. ANEMLL weights are evaluated in PyTorch; GGUF files [36, 37] are evaluated with llama.cpp [35]. The reported BF16 GGUF check differs from the PyTorch reference by at most 0.0004 in KL. WikiText uses 16 sequences of 1,024 tokens. The long-context score uses the last 2,048 tokens of 16 windows of 16,384 tokens. Table 2 values should therefore be compared within the table, rather than directly against the compiled KL-512 values.

Runtime measurements use DFlash2 speculative decoding [21, 22]: the target verifies an anchor token plus seven drafted tokens per step. The varied-prompt result averages 24 chat and coding prompts. The fixed-prompt measurements use separate context entries and are reported separately because generated text and draft acceptance affect throughput. Compiled ANE memory counts target chunks and the head, excluding the drafter and other process or system allocations.

Vector lookup table microbenchmark

Table 3 uses the repository's 25 September 2026 Core ML sweep [42–44], separate from the release's Core AI language-model benchmarks. The systems were an M5 Max (Mac17,6, macOS 27.2 build 26B5091g) and an M6 (Mac18,5, macOS 27.0 build 26A428). Models use the iOS 26 MIL target; the M5 Max report records coremltools 9.0. Each model chains 8 or 16 convolutions with 4,096 input and output channels, 1 × 1 kernels and a 4 × 4 spatial input. Balanced synthetic codebooks keep outputs finite. Inputs and activations are FP16; INT8 labels describe weights or LUT values.

The harness initializes every input element, performs 10 warmups, then five rounds of 50 synchronous predictions, alternating which chain length runs first. The per-layer slope is (median time for 16 layers − median time for 8 layers) / 8, reducing fixed call overhead. Published placement checks assign every convolution to NeuralEngine, and output cosine similarity rounds to 1.00000 against each quantized-weight reference. These checks validate execution of the synthetic models, not language-model accuracy. Table 3 is calculated from the archived timing files [43]; the historical experiments were not rerun for this manuscript.

6 Results

Compiled distributional fidelity

Across 64 recorded conversation sequences with 40,023 scored next-token positions, the compiled M6 packages measured mean KL-512 of 0.0518, median 0.009 and p99 0.50, with 91.9% top-1 agreement with BF16 (first release: 0.1838, 0.030, 2.28 and 86.0%). The M5 Max functions measured 0.0517. Both chips were within 0.0003 of the same-weight PyTorch result, 0.0515. These differences compare each model's KL to BF16; they are not direct pairwise KL values between the ANE and PyTorch outputs [26].

Comparison with other quantizations

Both GGUF baselines, Unsloth UD-IQ3_XXS and Draw Things IQ3_XXS, use fine-grained block quantization with local scales [35–37]. This block scaling is not supported by the ANE execution path used here [40, 41]. ANEMLL instead uses one learned vector codebook per MLP matrix with per-output-channel scales. Table 2 therefore compares fidelity and storage under different hardware constraints; these GGUF formats are not directly executable ANE alternatives.

Table 2 reports 13.4% lower development KL and 13.2% lower held-out KL than Unsloth UD-IQ3_XXS, and 31.2% lower KL on the measured 16K windows. The percentages are relative reductions in KL, not improvements in task accuracy. UD-IQ3_XXS has lower WikiText KL, 0.0506 versus 0.0707. Both representations round to 3.17 bits per base weight before ANEMLL's correction factors are counted. Including those factors, ANEMLL uses 9.74 GiB versus 9.46 GiB, approximately 3% more storage. Thus the comparison is close in storage cost but not exactly size-matched.

Table 2. Reported KL to BF16 under the cross-format protocol in Section 5. All storage values exclude embeddings. The ANEMLL row shows total packed weights including 0.30 GiB of FP16 correction factors; its factor-exclusive size is 9.44 GiB, or 3.17 bits per weight. Lower KL is better. GGUF artifact families are identified in [36, 37].

Quantization	GiB including factors	Bits per base weight	Dev KL	Held-out KL	WikiText KL	16K window KL
ANEMLL release 0.2	9.74	3.27	0.0536	0.0565	0.0707	0.762
Unsloth UD-IQ3_XXS	9.46	3.17	0.0619	0.0651	0.0506	1.108
Draw Things IQ3_XXS	8.94	3.00	0.0789	0.0833	0.1246	1.204

The release guide reports development KL of 0.1512 for release 0.1, 0.0897 after improved GPTQ and QAT with the earlier allocation, 0.0730 for a mixed two- and four-bit comparison, and 0.0536 for release 0.2 [26]. The mixed-bit experiment upgrades six late MLP layers from two-bit vector tables to four-bit scalar tables, leaving 32 layers at each bitrate. It uses improved GPTQ fitting and 600 QAT steps on the expanded training corpus. These stages combine changes in fitting, allocation and rotations; the matched comparisons below address specific changes.

Memory and throughput

Packed weights excluding embeddings take 9.44 GiB, 3.17 bits per weight over 25.6 billion weights; the rank-64 factors add 0.30 GiB, for 9.74 GiB in total (3.27 bits per weight). Compiled M6 ANE memory for chunks plus head is 12.97 GB (decimal gigabytes; 9.74 GiB is 10.46 GB), against 13.32 GB for the first release, excluding the rest of the process and runtime. These quantities describe different representations; neither is total machine memory or the .safetensors download size.

With DFlash2, the M6 averages 37.5 tokens/s over the 24-prompt corpus, versus 37.6 for the reported comparison using the earlier mixed formats. The M5 Max averages 16.6 tokens/s versus 17.2 for the separately fitted model with 32 two-bit and 32 four-bit MLP layers, a 3.5% decrease. The decrease is consistent with the measured rotation overhead, but end-to-end throughput also depends on acceptance [26].

On the fixed benchmark prompt, release 0.2 accepts fewer drafted tokens: 79% at the 8K entry, versus 87–89% for earlier weights. The reported 16K decode rate decreases from 60.3 to 53.3 tokens/s, an 11.6% reduction. These workload-specific outcomes explain why the varied-prompt average cannot establish equal speed for all prompts or context lengths.

M5 Max and M6 vector lookup table sweep

The synthetic sweep isolates the weight-format effect more directly than full-model generation. For the INT8 vector-LUT rows in Table 3, M6 is 2.23–4.88 times faster than M5 Max. The two-component, 16-entry format is 3.96 times faster, whereas dense INT8 differs by only 1.06 times. On M5 Max, LUT latency stays near 0.108–0.112 ms per layer from four to one-half nominal index bits per weight. FP16-codebook controls show the same qualitative chip difference, including a 4.34-fold speedup for the two-component, 16-entry table.

Table 3. Archived Core ML convolution sweep on M5 Max and M6 [42–44]. Lower time is better. In a vector LUT d × K, one index selects a d-component vector from K entries; nominal index bits per weight are $\log_2(K)/d$, excluding codebooks and scales. INT8 and FP16 specify the codebook value type for LUT rows. Speedup is M5 Max time divided by M6 time, calculated before rounding. These are per-layer microbenchmark slopes, not token-generation speeds.

Weight representation	Nominal bits/weight	M5 Max ms/layer	M6 ms/layer	M6 speedup over M5 Max
FP16 dense	16	0.211	0.218	0.97×
INT8 dense	8	0.109	0.103	1.06×
INT8 scalar LUT 16	4	0.108	0.045	2.42×
INT8 vector LUT 2 × 64	3	0.110	0.049	2.23×
INT8 vector LUT 2 × 16	2	0.112	0.028	3.96×
INT8 vector LUT 4 × 64	1.5	0.110	0.024	4.66×
INT8 vector LUT 4 × 16	1	0.111	0.026	4.33×
INT8 vector LUT 8 × 16	0.5	0.111	0.023	4.88×
INT8 vector LUT 16 × 16	0.25	0.125	0.030	4.12×
FP16 scalar LUT 16	4	0.112	0.055	2.02×
FP16 vector LUT 2 × 16	2	0.109	0.025	4.34×
FP16 vector LUT 4 × 16	1	0.111	0.026	4.26×

The gain is workload-dependent. On M6, scalar two-bit and vector 2 × 16 tables both take about 0.028 ms per layer, so VQ itself is not uniquely responsible for the lower latency at that index bitrate. In the separate 32-branch weight-streaming test, vector 2 × 16 takes 1.039 ms per call on M5 Max and 0.448 ms on M6, a 2.32-fold speedup; wider vectors do not improve M6 time further [42]. Placement and timing support accelerated VQ execution, but do not isolate the decoder microarchitecture. Different OS builds also limit attribution solely to the chip. The sweep measures neither task accuracy nor end-to-end LLM speedup, and its timing cannot determine six-bit packing.

Quantization and rotation ablations

Across 24 MLP matrices from eight sampled layers, three-bit tables close 80.3–82.8% of the Hessian-weighted output-error gap between two-bit and four-bit tables. No gate, up or down projection, or early versus late layer, stands out in that sample as requiring more bits. Normalized errors are approximately 1.00, 0.24 and 0.08 for two-, three- and four-bit formats. A half-two-bit, half-four-bit allocation would average 0.54 under this local-error calculation [26].

A matched 150-step QAT study compares three-bit tables in every MLP layer with two-bit tables in 32 layers and four-bit tables in the other 32. Both use the same seed, data and total MLP index budget. The all-three-bit allocation gives 23.0% lower development KL (reported interval 18.6–28.1%) and 22.7% lower selection KL (20.1–25.3%). This supports the tested allocation and training budget, rather than every possible mixed-bit design [26].

For the M6 8K verification entry, changing only MLP format adds 0.145 ms per layer for three-bit tables and 0.365 ms for scalar LUT4, relative to two-bit tables. On M5 Max, the tested formats differ by less than 0.2% in decode time. Scalar LUT4 down projections are duplicated in packages containing both decode and 64-row prefill functions; vector tables are shared. This compiler behavior helps explain the lower compiled memory despite extra index bits and rotations [26].

The estimated memory change relative to release 0.1 is +401 MB for index bits, about +0.4 GB for mixer rotations and −1,160 MB from removing duplicated down projections, or roughly −0.36 GB. The observed change is −0.35 GB. The corresponding format-only decode estimate is 64 × 0.145 = 9.28 ms per step, compared with 26 × 0.365 = 9.49 ms for the earlier allocation. These estimates describe format and compiler costs, not complete speculative generation latency [26].

Mixer rotation screening reduces local projection error by roughly 21–38%. In matched 600-step runs with two seeds per condition on the u15 proxy, rotations lower held-out KL by 6.4% (reported interval 3.2–9.4%) and sealed-trace KL by 7.6% (4.8–10.2%). On 16K windows the overall change is +2.2% (−0.3% to +5.5%), so a long-context benefit is not established. The u15 proxy raises the first release's 15 late two-bit MLP layers to four bits [26].

Online mixer transforms add approximately 1.2% decode time and up to 1.1% prefill time per chunk on M6, about 2–3% decode time on M5 Max, and about 0.4 GB of compiled target memory [26].

We also tested a QuaRot-style rotation folded into the residual stream. It required folding norm gains into the weights, which worsened MLP gate/up quantization error on the u15 proxy: 8% overall and 21–25% on its two-bit layers, exceeding the benefit recovered by the rotation. We kept the online transforms. The failure concerns quantization after norm-gain folding; an exact, consistently applied rotation does not itself distort the model. A separate SmoothQuant-style [9] input-channel scaling diagonal before the rotations was neutral to worse, apart from a 2–5% gain on the output projections that was too small to add. This unused input rescaling is distinct from the per-output-channel weight scales retained throughout the quantized projections [26].

7 Limitations

The quality evidence concerns distributional fidelity on fixed contexts. QAT minimizes a coarsened KL objective close to the evaluation metric, so lower KL is an expected training outcome and does not establish better coding, reasoning or instruction-following accuracy. The WikiText comparison also shows that the improvement does not hold across all evaluated distributions. Standard task benchmarks remain future work.

The released weights use one training seed. The rotation study has two seeds on a proxy allocation, and the matched three-bit study uses only 150 QAT steps. Those intervals do not quantify variation across independent end-to-end release training runs. Complete statistical reproduction additionally requires the underlying trace identities, interval-construction method and per-run records.

Packing, weight duplication, placement and timing depend on the tested shapes, compiler and OS.

The throughput comparisons are specific to their prompts, context entries, generated outputs and draft acceptance. Exact GGUF file revisions, backend build identifiers and complete timing configurations are needed to replay the cross-format and speed comparisons. Model-repository links alone are insufficient to fix those conditions.

At a 64K prompt with the drafter, the recorded 32 GB M6 setup needs about 2 GB of swap headroom. Packed parameter storage and compiled target memory therefore must not be presented as whole-system serving memory. Energy per token, including a matched GPU comparison, remains unmeasured. Private calibration sessions, nonpublic evaluation traces and incomplete historical environment records limit exact reproduction, as detailed next.

8 Code and data availability

The public source repository is ANEMLL Forge [25]. All ANEMLL GitHub file references point to the source revision identified in [25]. The main reproduction entry points are the quantization guide [26], sequential quantizer and LUT helpers [27], Core AI converter [28], release allocation [29], compiled KL evaluator [30], environment guide [31] and inference/download guide [32]. The bibliography gives revision identifiers and full URLs for use in both electronic and printed copies.

The hardware-format constraints and M5 Max/M6 sweep are documented in [41, 42]. Archived per-round timing medians are linked in [43], with the synthetic-model generator and synchronous timing harness in [44]. Those files support recomputing Table 3 without rerunning a benchmark. The sweep reports also link placement checks and the parallel-branch timing files.

The quantized safetensors export [33] contains indices, tables, scales, rotation metadata and factors. It is an input to conversion, not a directly loadable Transformers checkpoint. Its EXPORT.md explains the conversion procedure. The prepared inference bundle [34] contains Core AI target packages, the matching DFlash2 drafter and model assets. The drafter was rebuilt with the retrained target head; Core ML packages are a separate experimental route.

The cited public revision includes the mixer-rotation GPTQ implementation [27], frozen-index QAT [46], corpus generation and checked prompt list [47], and overlay materialization [48]. The training guide [45] provides commands, dependencies and source provenance. This code was recovered from a committed research snapshot; the record does not establish the exact complete checkout executed by the historical Colab job.

The public snapshot includes subsequent numerical safeguards and logging fixes, and offers FP32 teacher caching for new experiments. FP16 remains the historical default. These updates were checked with small CPU tests; the reported 27B training and benchmark results have not been rerun with them [45].

Private coding-agent calibration sessions are not distributed. Replacing them with public data defines a new experiment. The original cached development and selection traces are also not distributed. Exact retraining requires the historical runtime versions, source state, seeds, data splits and trace identifiers. Running the prepared release and recreating the training run are therefore different reproducibility goals.

For inference, use the complete matching target/drafter bundle and follow [32]. Bundle integrity checks and an inference smoke test verify installation and execution; they do not substitute for capability evaluation.

9 Conclusion

ANEMLL Forge release 0.2 combines three-bit MLP vector tables, online rotations and fixed-code QAT in a representation supported by the tested ANE path. Across the same 64 recorded conversation sequences, compiled mean KL-512 decreases from 0.1838 to 0.0518 and top-1 agreement increases from 86.0% to 91.9%, while compiled target memory decreases from 13.32 to 12.97 GB. The 24-prompt M6 decode average is 37.5 tokens/s, with workload-dependent slowdowns on other comparisons. The results support selecting quantization formats from measured compiler behavior and validating the compiled model. Broader task evaluation, energy measurements and a complete public training record remain necessary to assess the method beyond the present fidelity and deployment measurements.

References

[1] Frantar, Ashkboos, Hoefler and Alistarh. GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers. ICLR 2023. arXiv:2210.17323. https://arxiv.org/abs/2210.17323

[2] van Baalen et al. GPTVQ: The Blessing of Dimensionality for LLM Quantization. 2024. arXiv:2402.15319. https://arxiv.org/abs/2402.15319

[3] Egiazarian et al. Extreme Compression of Large Language Models via Additive Quantization (AQLM). ICML 2024. arXiv:2401.06118. https://arxiv.org/abs/2401.06118

[4] Liu et al. VPTQ: Extreme Low-bit Vector Post-Training Quantization for Large Language Models. EMNLP 2024. arXiv:2409.17066. https://arxiv.org/abs/2409.17066

[5] Chee, Cai, Kuleshov and De Sa. QuIP: 2-Bit Quantization of Large Language Models With Guarantees. NeurIPS 2023. arXiv:2307.13304. https://arxiv.org/abs/2307.13304

[6] Tseng et al. QuIP#: Even Better LLM Quantization with Hadamard Incoherence and Lattice Codebooks. ICML 2024. arXiv:2402.04396. https://arxiv.org/abs/2402.04396

[7] Ashkboos et al. QuaRot: Outlier-Free 4-Bit Inference in Rotated LLMs. NeurIPS 2024. arXiv:2404.00456. https://arxiv.org/abs/2404.00456

[8] Liu et al. SpinQuant: LLM Quantization with Learned Rotations. ICLR 2025. arXiv:2405.16406. https://arxiv.org/abs/2405.16406

[9] Xiao et al. SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models. ICML 2023. arXiv:2211.10438. https://arxiv.org/abs/2211.10438

[10] Kim et al. SqueezeLLM: Dense-and-Sparse Quantization. ICML 2024. arXiv:2306.07629. https://arxiv.org/abs/2306.07629

[11] Cho, Vahid, Adya and Rastegari. DKM: Differentiable K-Means Clustering Layer for Neural Network Compression. ICLR 2022. arXiv:2108.12659. https://arxiv.org/abs/2108.12659

[12] Yao et al. ZeroQuant-V2: Exploring Post-training Quantization in LLMs from Comprehensive Study to Low Rank Compensation. 2023. arXiv:2303.08302. https://arxiv.org/abs/2303.08302

[13] Li et al. LoftQ: LoRA-Fine-Tuning-Aware Quantization for Large Language Models. ICLR 2024. arXiv:2310.08659. https://arxiv.org/abs/2310.08659

[14] Malinovskii et al. PV-Tuning: Beyond Straight-Through Estimation for Extreme LLM Compression. NeurIPS 2024. arXiv:2405.14852. https://arxiv.org/abs/2405.14852

[15] Chen et al. EfficientQAT: Efficient Quantization-Aware Training for Large Language Models. ACL 2025. arXiv:2407.11062. https://arxiv.org/abs/2407.11062

[16] Liu et al. LLM-QAT: Data-Free Quantization Aware Training for Large Language Models. Findings of ACL 2024. arXiv:2305.17888. https://arxiv.org/abs/2305.17888

[17] Hinton, Vinyals and Dean. Distilling the Knowledge in a Neural Network. 2015. arXiv:1503.02531. https://arxiv.org/abs/1503.02531

[18] Dutta, Krishnan, Kwatra and Ramjee. Accuracy is Not All You Need. NeurIPS 2024. arXiv:2407.09141. https://arxiv.org/abs/2407.09141

[19] Gunter et al. Apple Intelligence Foundation Language Models. 2024. arXiv:2407.21075. https://arxiv.org/abs/2407.21075

[20] Yang, Kautz and Hatamizadeh. Gated Delta Networks: Improving Mamba2 with Delta Rule. ICLR 2025. arXiv:2412.06464. https://arxiv.org/abs/2412.06464

[21] Leviathan, Kalman and Matias. Fast Inference from Transformers via Speculative Decoding. ICML 2023. arXiv:2211.17192. https://arxiv.org/abs/2211.17192

[22] Chen, Liang and Liu. DFlash: Block Diffusion for Flash Speculative Decoding. ICML 2026. arXiv:2602.06036. https://arxiv.org/abs/2602.06036 Code: https://github.com/z-lab/dflash

[23] Apple Machine Learning Research. Deploying Transformers on the Apple Neural Engine. 2022. https://machinelearning.apple.com/research/neural-engine-transformers

[24] Qwen Team. Qwen3.8-27B model card and source checkpoint. Revision 1d4bf0f2ff6012fd82039f2fa52739d0dd7c60c0. https://huggingface.co/Qwen/Qwen3.8-27B/tree/1d4bf0f2ff6012fd82039f2fa52739d0dd7c60c0

[25] Anemll Open Source Project. ANEMLL Forge source code. 2026. Commit a13b4c7cb6ea548e05dfb900a671a96ac5107e4d. https://github.com/Anemll/anemll-forge/tree/a13b4c7cb6ea548e05dfb900a671a96ac5107e4d

[26] Anemll Open Source Project. Quantization of ANEMLL Forge Qwen3.8-27B. Release 0.2 methods, ablations and results. In [25]. https://github.com/Anemll/anemll-forge/blob/a13b4c7cb6ea548e05dfb900a671a96ac5107e4d/docs/QUANTIZATION.md

[27] Anemll Open Source Project. LUT fitting, encoding and GPTQ helpers; sequential quantizer with mixer rotations and low-rank fitting. In [25]. https://github.com/Anemll/anemll-forge/blob/a13b4c7cb6ea548e05dfb900a671a96ac5107e4d/scripts/qwen3_lut_common.py https://github.com/Anemll/anemll-forge/blob/a13b4c7cb6ea548e05dfb900a671a96ac5107e4d/scripts/qwen38_gptq_27b.py

[28] Anemll Open Source Project. Core AI model builder, online rotations and exact-table conversion. In [25]. https://github.com/Anemll/anemll-forge/blob/a13b4c7cb6ea548e05dfb900a671a96ac5107e4d/coreai/qwen38_coreai_build.py

[29] Anemll Open Source Project. Release 0.2 quantization allocation. In [25].
https://github.com/Anemll/anemll-forge/blob/a13b4c7cb6ea548e05dfb900a671a96ac5107e4d/configs/quantization/mix25in_vq3pA.json

[30] Anemll Open Source Project. Compiled Core AI KL-512 evaluation harness. In [25].
https://github.com/Anemll/anemll-forge/blob/a13b4c7cb6ea548e05dfb900a671a96ac5107e4d/scripts/m6_kl512_eval.py

[31] Anemll Open Source Project. Environment setup and provenance. In [25].
https://github.com/Anemll/anemll-forge/blob/a13b4c7cb6ea548e05dfb900a671a96ac5107e4d/docs/ENVIRONMENT.md

[32] Anemll Open Source Project. Model bundle download, integrity checks and inference guide. In [25].
https://github.com/Anemll/anemll-forge/blob/a13b4c7cb6ea548e05dfb900a671a96ac5107e4d/docs/HUGGING_FACE.md

[33] Anemll Open Source Project. Quantized Qwen3.8-27B export for Core AI and conversion instructions. Hugging Face, revision 21cd9aaa7ab54efc67acc37b627f7e63d42ca1c4.
https://huggingface.co/anemll/anemll-quantized-qwen3.8-27b-for-CoreAI/tree/21cd9aaa7ab54efc67acc37b627f7e63d42ca1c4
https://huggingface.co/anemll/anemll-quantized-qwen3.8-27b-for-CoreAI/blob/21cd9aaa7ab54efc67acc37b627f7e63d42ca1c4/EXPORT.md

[34] Anemll Open Source Project. ANEMLL Forge Qwen3.8-27B prepared inference bundle. Hugging Face, revision 440f4e366b43d0f9bbccd79e695b159c4a94d1a0.
https://huggingface.co/anemll/anemll-forge-qwen3.8-27B/tree/440f4e366b43d0f9bbccd79e695b159c4a94d1a0

[35] llama.cpp contributors. llama.cpp inference framework and IQ3 block-format definitions. GitHub. Accessed 13 October 2026.
https://github.com/ggml-org/llama.cpp
https://github.com/ggml-org/llama.cpp/blob/master/ggml/src/ggml-common.h

[36] Unsloth. Qwen3.8-27B GGUF quantizations, UD-IQ3_XXS artifact family. Hugging Face.
https://huggingface.co/unsloth/Qwen3.8-27B-GGUF

[37] Draw Things. Qwen3.8-27B GGUF quantizations, IQ3_XXS artifact family. Hugging Face.
https://huggingface.co/drawthingsai/Qwen3.8-27B-GGUF

[38] Open Compute Project. OCP Microscaling Formats (MX) Specification, version 1.0. September 2023.
https://www.opencompute.org/documents/ocp-microscaling-formats-mx-v1-0-spec-final-pdf

[39] Eduardo Alvarez et al. Introducing NVFP4 for Efficient and Accurate Low-Precision Inference. NVIDIA Technical Blog, 24 June 2025.
https://developer.nvidia.com/blog/introducing-nvfp4-for-efficient-and-accurate-low-precision-inference/

[40] Apple. Quantization Performance, Guide to Core ML Tools; Bring your machine learning and AI models to Apple silicon, WWDC 2024. Hardware-specific guidance on block quantization and grouped palettization. Accessed 12 October 2026.
https://apple.github.io/coremltools/docs-guides/source/opt-quantization-perf.html
https://developer.apple.com/videos/play/wwdc2024/10159/

[41] Anemll Open Source Project. Vector LUT investigation and observed ANE placement constraints. Historical research notes. In [25].
https://github.com/Anemll/anemll-forge/blob/a13b4c7cb6ea548e05dfb900a671a96ac5107e4d/docs/history/VECTOR_LUT_README.md

[42] Anemll Open Source Project. M6 INT8 vector LUT tests compared with M5 Max; M5 Max INT8 vector LUT tests. Measurements dated 25 September 2026. In [25].
https://github.com/Anemll/anemll-forge/blob/a13b4c7cb6ea548e05dfb900a671a96ac5107e4d/RESULTS_M6_INT8.md
https://github.com/Anemll/anemll-forge/blob/a13b4c7cb6ea548e05dfb900a671a96ac5107e4d/RESULTS_M5_MAX_INT8.md

[43] Anemll Open Source Project. Archived M6 and M5 Max LUT timing data, including per-round medians. In [25].
https://github.com/Anemll/anemll-forge/blob/a13b4c7cb6ea548e05dfb900a671a96ac5107e4d/results/m6_lut_timing.tsv
https://github.com/Anemll/anemll-forge/blob/a13b4c7cb6ea548e05dfb900a671a96ac5107e4d/results/m5_max_lut_timing.tsv

[44] Anemll Open Source Project. Synthetic LUT model generator and matched-chain Core ML timing harness. In [25].
https://github.com/Anemll/anemll-forge/blob/a13b4c7cb6ea548e05dfb900a671a96ac5107e4d/scripts/lut_fp8_coreml.py
https://github.com/Anemll/anemll-forge/blob/a13b4c7cb6ea548e05dfb900a671a96ac5107e4d/scripts/time_coreml_pair.swift

[45] Anemll Open Source Project. Release 0.2 training workflow, dependencies, validation and source provenance. In [25].
https://github.com/Anemll/anemll-forge/blob/a13b4c7cb6ea548e05dfb900a671a96ac5107e4d/docs/QUANTIZATION_TRAINING.md
https://github.com/Anemll/anemll-forge/blob/a13b4c7cb6ea548e05dfb900a671a96ac5107e4d/docs/quantization-source-provenance.json

[46] Anemll Open Source Project. Frozen-index quantization-aware training and teacher caching. In [25].
https://github.com/Anemll/anemll-forge/blob/a13b4c7cb6ea548e05dfb900a671a96ac5107e4d/scripts/qwen38_qat_kl.py

[47] Anemll Open Source Project. QAT corpus generation and checked release prompt list. In [25].
https://github.com/Anemll/anemll-forge/blob/a13b4c7cb6ea548e05dfb900a671a96ac5107e4d/scripts/qwen38_corpus_gen.py
https://github.com/Anemll/anemll-forge/blob/a13b4c7cb6ea548e05dfb900a671a96ac5107e4d/configs/kl/qat_corpus_prompts.json

[48] Anemll Open Source Project. QAT overlay materialization into a complete weight export. In [25].
https://github.com/Anemll/anemll-forge/blob/a13b4c7cb6ea548e05dfb900a671a96ac5107e4d/scripts/qwen38_overlay_materialize.py

[49] Anemll Open Source Project. ANE HWX and MLIR analysis. Compiled INT8/FP8 fusion and attention dataflow. In [25].
https://github.com/Anemll/anemll-forge/blob/a13b4c7cb6ea548e05dfb900a671a96ac5107e4d/docs/ANE_HWX_MLIR_ANALYSIS.md

[50] Anemll Open Source Project. M6 compute acceleration. Dated experiments on INT8 projection kernels, 8-bit softmax, FP8 range control and M5 variants. In [25].
https://github.com/Anemll/anemll-forge/blob/a13b4c7cb6ea548e05dfb900a671a96ac5107e4d/docs/research/M6_COMPUTE_ACCELERATION_2026-10-03.md

[51] Anemll Open Source Project. Release 0.2 validation on Apple M5; softmax probes and captured-attention device checks. In [25].
https://github.com/Anemll/anemll-forge/blob/a13b4c7cb6ea548e05dfb900a671a96ac5107e4d/docs/VALIDATION_M5_RELEASE_0_2.md
https://github.com/Anemll/anemll-forge/blob/a13b4c7cb6ea548e05dfb900a671a96ac5107e4d/scripts/m6_softmax_probe.py
https://github.com/Anemll/anemll-forge/blob/a13b4c7cb6ea548e05dfb900a671a96ac5107e4d/scripts/m6_attn_core_check.py

[52] Zhang, Huang, Zhang, Wei, Zhu and Chen. SageAttention2: Efficient Attention with Thorough Outlier Smoothing and Per-thread INT4 Quantization. ICML 2025. arXiv:2411.10958v7.
https://arxiv.org/abs/2411.10958v7

[53] Anemll Open Source Project. Verifier block length: 8, 4 and 3 tokens on the M6 ANE. Measured call times and modeled decode throughput, 3 October 2026. In [25].
https://github.com/Anemll/anemll-forge/blob/a13b4c7cb6ea548e05dfb900a671a96ac5107e4d/docs/verifier_len.md

[54] Anemll Open Source Project. Speculative server and Core AI target runtime: windowed drafter ingestion, accepted-prefix commit and reusable plans. In [25].
https://github.com/Anemll/anemll-forge/blob/a13b4c7cb6ea548e05dfb900a671a96ac5107e4d/scripts/qwen38_server.py
https://github.com/Anemll/anemll-forge/blob/a13b4c7cb6ea548e05dfb900a671a96ac5107e4d/scripts/qwen38_coreai_model.py

[55] Anemll Open Source Project. Core AI DFlash2 builder and runtime: fused context updates, host-managed ring and candidate selection. In [25].
https://github.com/Anemll/anemll-forge/blob/a13b4c7cb6ea548e05dfb900a671a96ac5107e4d/coreai/dflash2_coreai_build.py
https://github.com/Anemll/anemll-forge/blob/a13b4c7cb6ea548e05dfb900a671a96ac5107e4d/scripts/dflash2_coreai_drafter.py

[56] Yang, Wang, Zhang, Shen and Kim. Parallelizing Linear Transformers with the Delta Rule over Sequence Length. NeurIPS 2024. arXiv:2406.06484.
https://arxiv.org/abs/2406.06484

[57] Anemll Open Source Project. KV-cache formats and per-token, per-head INT8 quantization with FP16 scales. In [25].
https://github.com/Anemll/anemll-forge/blob/a13b4c7cb6ea548e05dfb900a671a96ac5107e4d/scripts/qwen38_kv_cache.py